

# Revisiting the Weerakoon–Fernando Variant of Newton’s Method: Replication, Extensions, and a Critical Assessment of Convergence Order

CSE 402 (Section C - Group 11) Project Report

<https://github.com/Pulseparadigm/CSE-402-Project.git>

Md. Shadman Abid

Student ID: 2105124

Bangladesh University of Engineering  
and Technology  
Dhaka, Bangladesh

Md. Yousuf Niaz

Student ID: 2105125

Bangladesh University of Engineering  
and Technology  
Dhaka, Bangladesh

Nusrat Jahan Tamanna

Student ID: 2105140

Bangladesh University of Engineering  
and Technology  
Dhaka, Bangladesh

K. M. Sakib Hasan

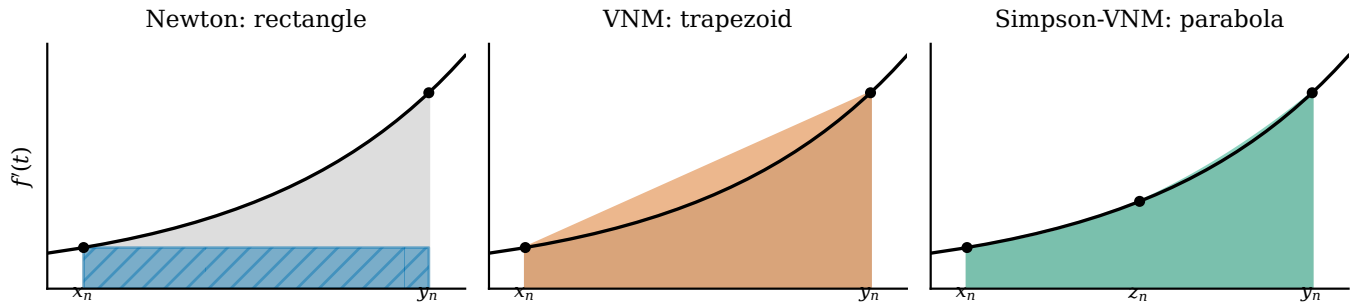
Student ID: 2105157

Bangladesh University of Engineering  
and Technology  
Dhaka, Bangladesh

Md. Shaidur Rahman Pranto

Student ID: 2105177

Bangladesh University of Engineering  
and Technology  
Dhaka, Bangladesh



**Figure 1: Geometric idea behind the methods studied.** Each method approximates the area under  $f'(t)$  between the current iterate  $x_n$  and the (predicted) next iterate  $y_n$ : Newton uses a rectangle, VNM a trapezoid, and S-VNM a parabola through the midpoint  $z_n$ .

## Abstract

Newton’s method (NM) is the standard tool for solving  $f(x) = 0$ , but it converges only quadratically. Weerakoon and Fernando (2000) showed that replacing the rectangle approximation of the integral in Newton’s theorem with a trapezoid yields a two-step variant (VNM) of third order that needs no second derivatives. In this project we implement NM and VNM in Python, replicate the base paper’s benchmark of 18 test cases, and build three extensions: a Simpson-rule variant (S-VNM), a derivative-free Steffensen-type variant (St-VNM), and a multivariate VNM for nonlinear systems. We assess all methods with iteration counts, function evaluations (NFEV), the computational order of convergence (COC) in double and 400-digit arithmetic, and symbolic Taylor-series error analysis. The replication confirms the paper: VNM attains a COC of 3.000 and its error constant  $C_2^2 + \frac{1}{2}C_3$  agrees with the measurements to six significant digits; our iteration counts match the published ones exactly in 13 of 18 cases (VNM). Contrary to our initial design goals, S-VNM is third order ( $e_{n+1} = C_2^2 e_n^3$ ), not fourth, and St-VNM is

only second order, because the Newton predictor and the finite-difference slope, respectively, limit accuracy. The multivariate VNM is third order and needs 3 iterations versus 5 for Newton. We also identify a typographical error in one of the paper’s test functions and show that most of VNM’s NFEV saving comes from difficult starting points.

## CCS Concepts

• **Mathematics of computing** → **Nonlinear equations**; *Numerical analysis*; Mathematical software performance.

## Keywords

Newton’s method, nonlinear equations, third-order convergence, quadrature, computational order of convergence, Steffensen’s method, nonlinear systems

## 1 Introduction and Base Paper Review

### 1.1 Background

Finding a simple root  $\alpha$  of a nonlinear equation  $f(x) = 0$  is one of the oldest problems in numerical analysis [1, 11, 16]. Newton's method,

$$x_{n+1} = x_n - \frac{f(x_n)}{f'(x_n)}, \quad (1)$$

converges quadratically near  $\alpha$ : the number of correct digits roughly doubles at every step. Methods of higher order exist (Halley, Chebyshev), but classical ones need  $f''$ , which may be expensive or unavailable. This motivated a family of *multipoint* methods that reach higher order using only  $f$  and  $f'$  evaluated at a few points [9, 16]. Two quantities matter for comparing such methods: the order  $p$ , and the number  $m$  of function evaluations per iteration. They combine into the efficiency index  $E = p^{1/m}$  [11]. Kung and Traub [9] conjectured that a method without memory using  $m$  evaluations can have order at most  $2^{m-1}$ .

### 1.2 The Base Paper

Our selected base paper is S. Weerakoon and T. G. I. Fernando, "A Variant of Newton's Method with Accelerated Third-Order Convergence," *Applied Mathematics Letters* 13(8):87–93, 2000 [18]. Its key observation is that Newton's method can be derived from the identity (Newton's theorem)

$$f(x) = f(x_n) + \int_{x_n}^x f'(t) dt, \quad (2)$$

by approximating the integral with the *rectangle*  $f'(x_n)(x - x_n)$  (Figure 1, left). The authors instead use the *trapezoid*  $\frac{1}{2}(x - x_n)[f'(x_n) + f'(x)]$ , which gives a local model that also matches the curvature of  $f$  at  $x_n$ . Setting the model to zero produces an implicit scheme, which they make explicit by predicting the unknown endpoint with a Newton step. The resulting variant of Newton's method (VNM) is

$$y_n = x_n - \frac{f(x_n)}{f'(x_n)}, \quad x_{n+1} = x_n - \frac{2f(x_n)}{f'(x_n) + f'(y_n)}. \quad (3)$$

Via a Taylor expansion the paper proves (their Eq. 4.9) that, for a simple root and  $f \in C^3$ ,

$$e_{n+1} = \left(C_2^2 + \frac{1}{2}C_3\right)e_n^3 + O(e_n^4), \quad C_j = \frac{f^{(j)}(\alpha)}{j! f'(\alpha)}, \quad (4)$$

where  $e_n = x_n - \alpha$ . VNM therefore converges cubically with three evaluations per step ( $f(x_n), f'(x_n), f'(y_n)$ ), giving  $E = 3^{1/3} \approx 1.442$  against  $2^{1/2} \approx 1.414$  for Newton. Numerical tests on nine functions (their Table 1) report a COC close to 3 for VNM and typically fewer total function evaluations (NOFE) than NM. The paper helped start a large literature on quadrature-based Newton variants [2, 3, 6, 12], including extensions to systems [4].

### 1.3 Objectives of This Work

Our project has four objectives:

- (1) **Implement and replicate.** Implement NM and VNM and reproduce the paper's Table 1 (18 function/starting-point cases).
- (2) **Extend.** Design and implement three extensions: a Simpson-rule variant (S-VNM), intended to raise the order to four; a derivative-free Steffensen-type variant (St-VNM), intended

---

#### Algorithm 1 Weerakoon–Fernando variant (VNM)

---

**Require:**  $f, f', x_0$ , tolerance  $\varepsilon$ , iteration cap  $N$

```

1: for  $n = 0, 1, \dots, N - 1$  do
2:    $f_n \leftarrow f(x_n); d_n \leftarrow f'(x_n)$  ▷ 2 evaluations
3:   if  $|d_n| < 10^{-14}$  then fail (flat slope)
4:   end if
5:    $y_n \leftarrow x_n - f_n/d_n$  ▷ Newton predictor
6:    $D \leftarrow d_n + f'(y_n)$  ▷ 1 evaluation
7:   if  $|D| < 10^{-14}$  then fail
8:   end if
9:    $x_{n+1} \leftarrow x_n - 2f_n/D$  ▷ trapezoidal corrector
10:  if  $|x_{n+1} - x_n| < \varepsilon$  or  $|f(x_{n+1})| < \varepsilon$  then
11:    return  $x_{n+1}$ 
12:  end if
13: end for
```

---

to keep third order without  $f''$ ; and a multivariate VNM for systems  $F(X) = 0$ .

- (3) **Benchmark.** Compare all methods on six test functions and one  $2 \times 2$  system using iterations, NFEV and COC.
- (4) **Verify.** Check every theoretical order claim with symbolic error analysis and high-precision experiments, and report where the claims fail.

The fourth objective led to the main finding of this report: two of our three extensions do not reach the order we initially expected. Section 2 explains why.

## 2 Methodology

All scalar methods start from Eq. (2). They differ in how the integral is approximated and in what information ( $f$  or  $f'$ ) is used. Table 1 summarises them.

### 2.1 Newton's Method (NM)

The rectangle rule gives Eq. (1). Using symbolic series expansion with SymPy [10] we recover the classical error equation

$$e_{n+1} = C_2 e_n^2 + 2(C_3 - C_2^2) e_n^3 + O(e_n^4). \quad (5)$$

### 2.2 Weerakoon–Fernando Variant (VNM)

The trapezoid rule together with a Newton predictor gives Eq. (3) (Algorithm 1). Our independent symbolic derivation reproduces the paper's error equation (4) exactly, and gives the next term as  $-\frac{1}{2}(6C_2^3 - 3C_2C_3 - 2C_4)e_n^4$ .

### 2.3 Extension 1: Simpson-Rule Variant (S-VNM)

Simpson's  $\frac{1}{3}$  rule is exact for cubics, whereas the trapezoid is exact only for lines. This motivated our first extension. With the midpoint  $z_n = \frac{1}{2}(x_n + y_n)$  the integral becomes  $\frac{1}{6}(x - x_n)[f'(x_n) + 4f'(z_n) + f'(y_n)]$ , giving

$$x_{n+1} = x_n - \frac{6f(x_n)}{f'(x_n) + 4f'(z_n) + f'(y_n)}, \quad (6)$$

with four evaluations per step. We initially expected fourth-order convergence. The symbolic analysis instead gives

$$e_{n+1} = C_2^2 e_n^3 - 3C_2(C_2^2 - C_3) e_n^4 + O(e_n^5), \quad (7)$$

so S-VNM is *third order*. The reason becomes clear when Eqs. (4) and (7) are compared. In VNM the term  $\frac{1}{2}C_3$  comes from the trapezoid's own quadrature error, and Simpson's rule removes it. The term  $C_2^2$ , however, is identical in both methods. It arises because  $f'$  is sampled at the *predicted* endpoint  $y_n$ , not at the true endpoint. Since  $y_n$  carries the Newton error  $C_2 e_n^2$ , this perturbation enters the denominator at  $O(e_n^2)$  and the new error at  $O(e_n^3)$ , however accurate the quadrature rule is. This agrees with the observation of Cordero and Torregrosa [2] that quadrature-based Newton variants with a Newton predictor remain third order. With four evaluations, S-VNM has  $E = 3^{1/4} \approx 1.316$ , lower than both NM and VNM.

## 2.4 Extension 2: Steffensen-Type Variant (St-VNM)

To remove the need for  $f'$ , we replace both derivatives in Eq. (3) by Steffensen divided differences [14]:

$$f'(x) \approx \frac{f(x + f(x)) - f(x)}{f(x)}. \quad (8)$$

This costs four evaluations of  $f$  per iteration:  $f(x_n)$ ,  $f(x_n + f(x_n))$ ,  $f(y_n)$  and  $f(y_n + f(y_n))$ . Our symbolic analysis gives

$$e_{n+1} = \frac{1}{2}f'(\alpha) C_2 e_n^2 + O(e_n^3), \quad (9)$$

so the method is *second order*, not third as we had intended. The cause is that Eq. (8) has an  $O(f(x_n)) = O(e_n)$  error. The trapezoidal corrector needs the slope at  $x_n$  to be accurate to  $O(e_n^2)$  to cancel the quadratic term, and this approximation cannot provide it. The constant also depends on the scale of  $f$  (through  $f'(\alpha)$ ), a known weakness of Steffensen-type methods. The efficiency index is  $2^{1/4} \approx 1.189$ . The advantage of St-VNM is that it is derivative-free; it is not faster.

## 2.5 Extension 3: Multivariate VNM for Systems

For  $F : \mathbb{R}^k \rightarrow \mathbb{R}^k$  with Jacobian  $J$ , the vector analogue of Eq. (2) is  $F(X) = F(X_n) + \int_0^1 J(X_n + t(X - X_n))(X - X_n) dt$ . The trapezoidal approximation and a Newton predictor give [4]

$$\begin{aligned} J(X_n) \Delta Y &= F(X_n), & Y_n &= X_n - \Delta Y, \\ (J(X_n) + J(Y_n)) \Delta X &= 2F(X_n), & X_{n+1} &= X_n - \Delta X. \end{aligned} \quad (10)$$

Each iteration evaluates  $F$  once and  $J$  twice, and solves two  $k \times k$  linear systems. We solve them by LU factorisation (scipy.linalg.solve) and never form  $J^{-1}$  explicitly. The method is third order [4], which we verify numerically in Section 4.

## 2.6 Performance Metrics

- **Iterations**  $i$  needed to meet the stopping test (Algorithm 1).
- **NFEV**, the total number of evaluations of  $f$  and  $f'$ , using the base paper's convention. The extra residual  $f(x_{n+1})$  computed only for the stopping test is not counted, as in [18].
- **COC**. The paper's definition uses errors relative to the root:  $\rho = \ln |e_{n+1}/e_n| / \ln |e_n/e_{n-1}|$ . Our double-precision benchmark uses the *approximated* COC (ACOC) [2], which replaces  $e_n$  by the step  $|x_n - x_{n-1}|$  and needs no knowledge of  $\alpha$ . For the high-precision experiments we use the true errors relative to a 400-digit reference root.

- **Asymptotic error constant**  $K = \lim |e_{n+1}|/|e_n|^p$ , compared with the theoretical values from Eqs. (4)–(9).

## 3 Implementation and Experiments

### 3.1 Software

All methods are implemented in a single Python 3 module, `weerakoon_fernando.py` (about 580 lines), in our repository [13]. We used NumPy [5], SciPy [17] (linear solves), and Matplotlib [7] (figures). For verification we additionally used SymPy [10] for the symbolic series expansions and mpmath [15] for arbitrary-precision arithmetic. Every scalar solver has the same interface and returns the root, the iterate history, the step sizes, the NFEV and the iteration count. The core of VNM is:

```
yn = x - fx / dfx # Newton predictor
denom = dfx + df(yn) # f'(x_n) + f'(y_n)
x_next = x - 2.0 * fx / denom # trapezoidal corrector
```

Every method guards against near-zero denominators (threshold  $10^{-14}$ ) and stops when  $|x_{n+1} - x_n| < \varepsilon$  or  $|f(x_{n+1})| < \varepsilon$ . The figures and the verification scripts are included with this report in the `code/` folder.

### 3.2 Test Problems (Benchmarks)

*Project benchmark (Experiment A)*. We use six scalar functions with simple roots, listed in Table 2, and the  $2 \times 2$  system

$$F(x, y) = \begin{pmatrix} x^2 + y^2 - 4 \\ xy - 1 \end{pmatrix} = 0, \quad X_0 = (1.5, 0.5)^T, \quad (11)$$

which has the exact solution

$$X^* = \left( \frac{\sqrt{6} + \sqrt{2}}{2}, \frac{\sqrt{6} - \sqrt{2}}{2} \right) \approx (1.931852, 0.517638).$$

Functions  $f_1$  and  $f_2$  are taken from the base paper, and  $f_1$  is started at the paper's difficult point  $x_0 = -0.5$ .

*Base-paper benchmark (Experiment B)*. The nine functions and 18 starting points of the paper's Table 1, listed in Table 3. While running this benchmark we found that function (8) as printed,  $x^2 \sin^2 x + e^{x^2 \cos x \sin x} - 28$ , does *not* vanish at the published root 4.82458931731526: the residual there is  $-4.94$ . The function  $x^2 \sin^2 x + e^{x^2 \cos x \sin x} - 28$  has a residual of  $5 \times 10^{-11}$  at that point, so we conclude that the printed product  $\cos x \sin x$  is a typographical error for a sum, and we use the corrected form.

### 3.3 Experimental Protocol and Parameters

**Experiment A (double precision)**. Our module is run unchanged, with tolerance  $\varepsilon = 10^{-15}$  and a cap of 100 iterations for scalar problems, and  $\varepsilon = 10^{-12}$  for the system. We report iterations, NFEV and ACOC.

**Experiment B (replication)**. NM and VNM follow exactly the same algorithms and stopping rule, with  $\varepsilon = 10^{-15}$ . The arithmetic is 60-digit (mpmath) so that roundoff cannot prevent termination. This matters here: in IEEE double precision the absolute test  $|x_{n+1} - x_n| < 10^{-15}$  is below the spacing of floating-point numbers near roots with  $|\alpha| > 4.5$ , and some runs stalled. The COC uses the paper's root-based definition.

**Experiment C (order verification)**. All four scalar methods are iterated in 400-digit arithmetic from the Experiment A

**Table 1: Summary of the methods studied.**  $m$  is the number of function evaluations per iteration under the base paper’s convention (each of  $f$  and  $f'$  counts as one). “Expected” is the order we aimed for when designing the extension; “verified” is the order established in this work by symbolic analysis and 400-digit experiments.

| Method                    | Update rule                               | Needs $f'$ ? | $m$       | Expected $p$ | Verified $p$ | $E = p^{1/m}$ |
|---------------------------|-------------------------------------------|--------------|-----------|--------------|--------------|---------------|
| NM (baseline)             | $x_n - f/f'(x_n)$                         | yes          | 2         | 2            | <b>2</b>     | 1.414         |
| VNM (base paper)          | $x_n - 2f/[f'(x_n) + f'(y_n)]$            | yes          | 3         | 3            | <b>3</b>     | <b>1.442</b>  |
| S-VNM (ext. 1)            | $x_n - 6f/[f'(x_n) + 4f'(z_n) + f'(y_n)]$ | yes          | 4         | 4            | <b>3</b>     | 1.316         |
| St-VNM (ext. 2)           | VNM with Steffensen slopes (8)            | no           | 4         | 3            | <b>2</b>     | 1.189         |
| Multivariate VNM (ext. 3) | $(J(X_n) + J(Y_n))\Delta X = 2F(X_n)$     | Jacobian     | $1F + 2J$ | 3            | <b>3</b>     | —             |

starting points. We measure the COC and  $K$  once the error lies between  $10^{-120}$  and  $10^{-8}$ , and compare with the symbolic predictions. For St-VNM we measure only while  $|e_n| > 10^{-150}$ , because below that the difference quotient (8) loses digits to cancellation. The multivariate NM and VNM are compared in the same way in 300-digit arithmetic.

## 4 Results and Discussion

### 4.1 Experiment A: Project Benchmark

Table 2 and Figures 2–3 summarise the double-precision benchmark.

*Iterations.* Both third-order methods need the fewest iterations on every function (Figure 2): 15 in total on  $f_2$ – $f_6$ , against 21 for NM and St-VNM. S-VNM never needs fewer iterations than VNM, which is consistent with both being third order. On  $f_1$  it needs more (10 versus 6), because its transient phase far from the root behaves differently.

*Function evaluations.* Per-iteration cost decides the NFEV comparison. On the well-behaved functions  $f_2$ – $f_6$ , NM uses 42 evaluations and VNM 45, a near tie as the similar efficiency indices (1.414 and 1.442) predict. VNM’s clear advantage appears on  $f_1$  from  $x_0 = -0.5$ , where  $f'(x) = 3x^2 + 8x$  nearly vanishes near  $x = 0$ . Newton’s first step lands at  $x_1 \approx -3.31$ , close to the local maximum of  $f$  at  $x = -8/3$ , and the iterates then oscillate around it for more than 100 steps. VNM uses the averaged slope  $\frac{1}{2}[f'(x_0) + f'(y_0)]$ , which sends the first iterate to  $x_1 \approx 5.37$ , to the right of the root. There  $f$  is increasing and convex, so the method then converges quickly: 6 steps and 18 evaluations in total. S-VNM costs about a third more than VNM throughout. St-VNM is the most expensive (79 evaluations on  $f_2$ – $f_6$ , 91 on  $f_1$ ), but it is the only method that needs no derivative.

*Double-precision COC.* The ACOC values in Table 2 lie near 2 for NM and St-VNM and near 3 for VNM and S-VNM. None is near 4. Double precision alone cannot settle the order question, however, because only two or three meaningful steps occur before roundoff. We therefore rely on Experiment C for that.

### 4.2 Experiment B: Replication of the Base Paper

Table 3 compares our replication with the paper. The main claims are confirmed:

- **Iteration counts.** We match the paper exactly in 13 of 18 cases for VNM and in 10 of 18 for NM. The counts are within

one iteration in 17 and 16 cases respectively. VNM needs fewer iterations than NM in all 18 cases, in both the paper and our runs. The remaining one-iteration differences are expected: the paper does not state its floating-point precision or the exact order of its two stopping tests.

- **Order.** Our root-based COC is 3.00 for VNM in 17 of 18 cases; the exception, 2.75, occurs where VNM stops after only two steps. The paper’s values (2.67–3.05) are noisier, which is typical of COC computed in limited precision. For NM we obtain 2.00 throughout. The paper’s low NM values for function (3) (1.56 and 1.66) are not reproduced, which suggests they come from precision loss.
- **Newton’s wandering on function (1).** The largest discrepancies (109 vs. 97 and 113 vs. 54 iterations) occur in the chaotic pre-asymptotic phase of Newton’s method. There, tiny arithmetic differences change which path the iterates take. The paper’s qualitative point, that NM needs roughly 100 iterations while VNM needs 6, is fully confirmed.

*Where VNM’s NFEV advantage comes from.* In total, VNM uses far fewer evaluations than NM: 228 versus 638 in the paper and 246 versus 508 in our runs. However, if the two difficult starting points of function (1) are excluded, the totals are practically equal: 192 versus 194 (paper) and 210 versus 206 (ours). Near the root, then, VNM’s cubic order roughly compensates for its extra evaluation per step, as the efficiency indices predict. Its large practical gain comes from greater robustness far from the root, when Newton’s step is erratic. We regard this as a more precise reading of the paper’s conclusion that VNM “requires fewer function evaluations.”

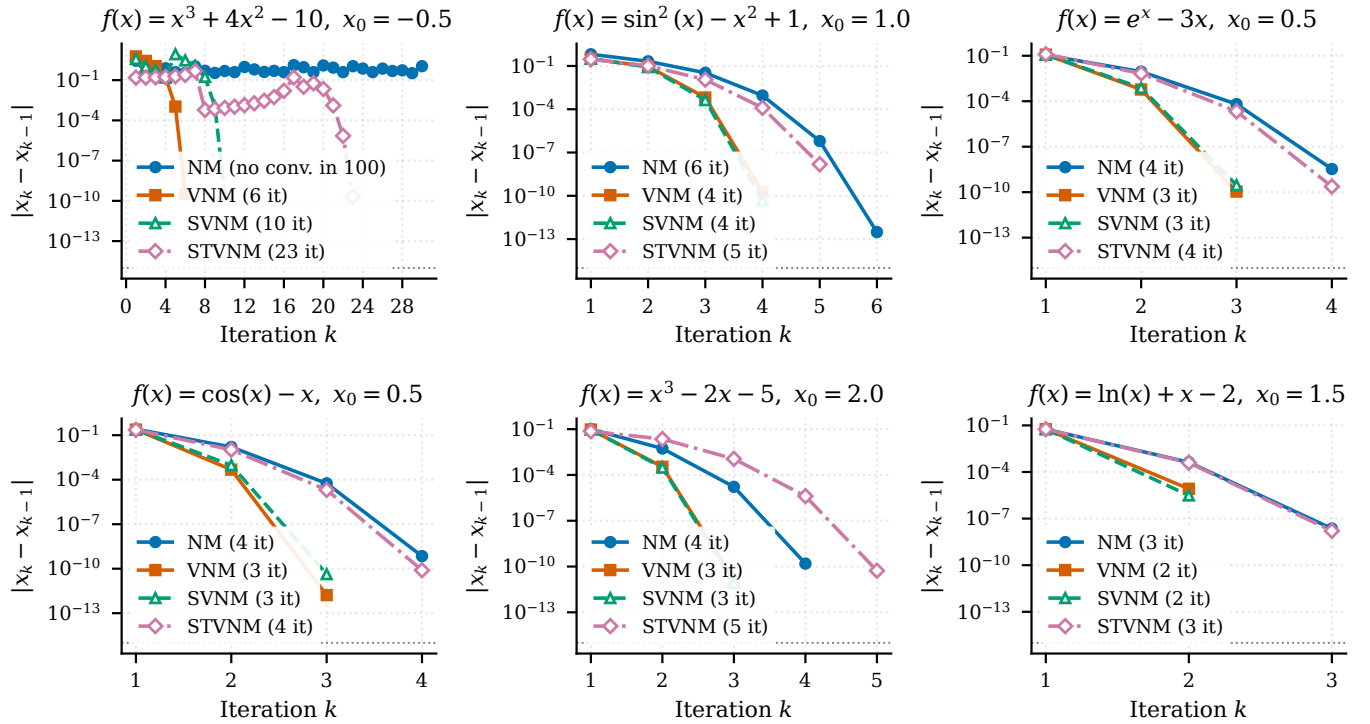
### 4.3 Experiment C: Verifying the Orders of the Extensions

Figure 4 and Table 4 settle the order question. In 400-digit arithmetic the COC of VNM and S-VNM converges to exactly 3.0000 and that of NM and St-VNM to exactly 2.0000, on all six functions. For every method and function, the measured constant  $K$  agrees with the symbolically derived constant to six significant digits. This confirms the base paper’s Eq. (4) and our own Eqs. (7) and (9), and shows that S-VNM is third order and St-VNM second order. Two further observations are worth noting:

- The S-VNM constant ( $C_2^2$ ) is smaller than the VNM constant ( $|C_2^2 + \frac{1}{2}C_3|$ ) whenever  $C_3 > 0$  ( $f_1, f_2, f_5, f_6$ ), and larger when

**Table 2: Experiment A (double precision,  $\varepsilon = 10^{-15}$ ): iterations  $i$ , function evaluations NFEV and approximated COC (ACOC) for each method.  $\dagger$ NM hit the 100-iteration cap without converging (the paper reports 109 iterations from this point). “–” means that fewer than three steps were available to compute the ACOC.**

|                         |                      | NM   |               |               | VNM  |           |           | S-VNM |           |      | St-VNM |      |      |      |
|-------------------------|----------------------|------|---------------|---------------|------|-----------|-----------|-------|-----------|------|--------|------|------|------|
| $f(x)$                  | $x_0$                | $i$  | NFEV          | ACOC          | $i$  | NFEV      | ACOC      | $i$   | NFEV      | ACOC | $i$    | NFEV | ACOC |      |
| $f_1$                   | $x^3 + 4x^2 - 10$    | -0.5 | $100^\dagger$ | $200^\dagger$ | –    | <b>6</b>  | <b>18</b> | 2.97  | 10        | 40   | 2.97   | 23   | 91   | 2.00 |
| $f_2$                   | $\sin^2 x - x^2 + 1$ | 1.0  | 6             | <b>12</b>     | 2.00 | <b>4</b>  | <b>12</b> | 3.05  | <b>4</b>  | 16   | 3.03   | 5    | 19   | 2.00 |
| $f_3$                   | $e^x - 3x$           | 0.5  | 4             | <b>8</b>      | 2.00 | <b>3</b>  | 9         | 2.93  | <b>3</b>  | 12   | 2.93   | 4    | 15   | 1.99 |
| $f_4$                   | $\cos x - x$         | 0.5  | 4             | <b>8</b>      | 2.00 | <b>3</b>  | 9         | 3.13  | <b>3</b>  | 12   | 3.07   | 4    | 15   | 2.00 |
| $f_5$                   | $x^3 - 2x - 5$       | 2.0  | 4             | <b>8</b>      | 2.00 | <b>3</b>  | 9         | 3.02  | <b>3</b>  | 12   | 3.02   | 5    | 19   | 2.00 |
| $f_6$                   | $\ln x + x - 2$      | 1.5  | 3             | <b>6</b>      | 2.01 | <b>2</b>  | <b>6</b>  | –     | <b>2</b>  | 8    | –      | 3    | 11   | 2.04 |
| Total ( $f_2$ – $f_6$ ) |                      |      | 21            | <b>42</b>     |      | <b>15</b> | 45        |       | <b>15</b> | 60   |        | 21   | 79   |      |



**Figure 2: Experiment A: step size  $|x_k - x_{k-1}|$  per iteration on a log scale (the dotted line marks  $10^{-15}$ ). The third-order methods (VNM, S-VNM) have steeper slopes than the second-order ones (NM, St-VNM), and the VNM and S-VNM curves nearly coincide. For  $f_1$  only the first 30 of the 100 non-converging Newton iterations are shown.**

$C_3 < 0$  ( $f_3, f_4$ ). Simpson’s rule therefore improves the *constant* for some functions, never the *order*, and it always costs an extra evaluation.

- The St-VNM constants for  $f_1$  (4.05) and  $f_5$  (3.14) are large because  $|f'(\alpha)|$  is large (16.5 and 11.2). This illustrates that Steffensen’s method is not invariant to the scaling of  $f$ .

#### 4.4 Multivariate VNM

On system (11) the multivariate VNM converges in 3 iterations (6 Jacobian and 3 residual evaluations) to

$$X_3 = (1.931851652578137, 0.517638090205042).$$

This agrees with the exact solution to machine precision. In double precision the ACOC is 2.89; in 300-digit arithmetic the errors decrease as  $4.3 \times 10^{-1}$ ,  $1.1 \times 10^{-2}$ ,  $2.6 \times 10^{-7}$ ,  $4.3 \times 10^{-21}$ ,  $2.0 \times 10^{-62}$ ,  $1.9 \times 10^{-186}$ , a COC of exactly 3.0 (Figure 5). Newton’s method needs 5 iterations to reach  $10^{-15}$  and shows COC 2.0. The multivariate

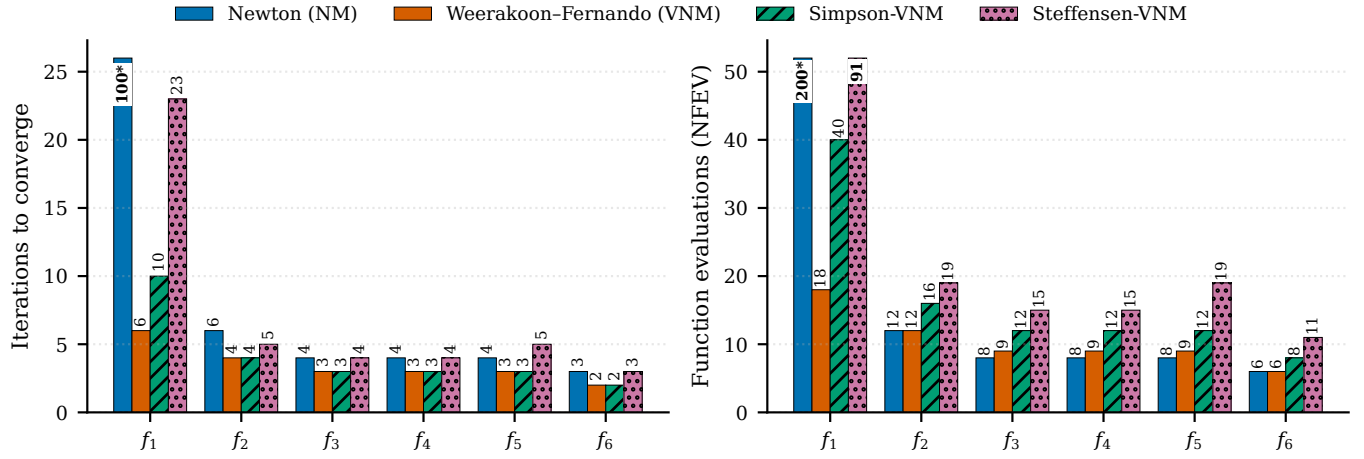


Figure 3: Experiment A: iterations (left) and function evaluations (right) per test function. Bars exceeding the axis are clipped and labelled with their true value; an asterisk marks a run that did not converge.

Table 3: Experiment B: replication of Table 1 of [18] (P = paper, O = ours; 60-digit arithmetic,  $\varepsilon = 10^{-15}$ , same stopping rule as our code). – = COC “not defined” in the paper. <sup>‡</sup>Function (8) with the typographical correction explained in Section 3. Bold marks cases where our iteration count differs from the paper’s.

| #                               | $f(x)$                                        | $x_0$ | $i$ (NM) |           | $i$ (VNM) |          | NOFE (NM) |     | NOFE (VNM) |     | COC (VNM) |      | Root $\alpha$     |
|---------------------------------|-----------------------------------------------|-------|----------|-----------|-----------|----------|-----------|-----|------------|-----|-----------|------|-------------------|
|                                 |                                               |       | P        | O         | P         | O        | P         | O   | P          | O   | P         | O    |                   |
| (1)                             | $x^3 + 4x^2 - 10$                             | -0.5  | 109      | <b>97</b> | 6         | 6        | 218       | 194 | 18         | 18  | 2.96      | 3.00 | 1.36523001341410  |
|                                 |                                               | 1     | 5        | 5         | 3         | 3        | 10        | 10  | 9          | 9   | –         | 3.00 |                   |
|                                 |                                               | 2     | 5        | 5         | 3         | <b>4</b> | 10        | 10  | 9          | 12  | –         | 3.00 |                   |
|                                 |                                               | -0.3  | 113      | <b>54</b> | 6         | 6        | 226       | 108 | 18         | 18  | 3.05      | 3.00 |                   |
| (2)                             | $\sin^2 x - x^2 + 1$                          | 1     | 5        | <b>6</b>  | 4         | 4        | 10        | 12  | 12         | 12  | 3.04      | 3.00 | 1.40449164821534  |
|                                 |                                               | 3     | 6        | 6         | 3         | 3        | 12        | 12  | 9          | 9   | –         | 3.00 |                   |
| (3)                             | $x^2 - e^x - 3x + 2$                          | 2     | 4        | <b>5</b>  | 4         | 4        | 8         | 10  | 12         | 12  | 3.01      | 3.00 | 0.25753028543986  |
|                                 |                                               | 3     | 6        | 6         | 4         | 4        | 12        | 12  | 12         | 12  | 3.04      | 3.00 |                   |
| (4)                             | $\cos x - x$                                  | 1     | 4        | 4         | 2         | 2        | 8         | 8   | 6          | 6   | –         | 2.75 | 0.73908513321516  |
|                                 |                                               | 1.7   | 4        | 4         | 3         | 3        | 8         | 8   | 9          | 9   | –         | 3.01 |                   |
|                                 |                                               | -0.3  | 5        | 5         | 3         | 3        | 10        | 10  | 9          | 9   | –         | 3.05 |                   |
| (5)                             | $(x - 1)^3 - 1$                               | 3.5   | 7        | 7         | 4         | <b>5</b> | 14        | 14  | 12         | 15  | 2.67      | 3.00 | 2                 |
|                                 |                                               | 2.5   | 5        | <b>6</b>  | 4         | 4        | 10        | 12  | 12         | 12  | 2.99      | 3.00 |                   |
| (6)                             | $x^3 - 10$                                    | 1.5   | 5        | <b>6</b>  | 4         | 4        | 10        | 12  | 12         | 12  | 3.01      | 3.00 | 2.15443469003188  |
| (7)                             | $xe^{x^2} - \sin^2 x + 3 \cos x + 5$          | -2    | 8        | 8         | 5         | <b>6</b> | 16        | 16  | 15         | 18  | 2.92      | 3.00 | -1.20764782713092 |
| (8) <sup>‡</sup>                | $x^2 \sin^2 x + e^{x^2 \cos x + \sin x} - 28$ | 5     | 9        | <b>10</b> | 5         | 7        | 18        | 20  | 15         | 21  | 2.87      | 3.00 | 4.82458931731564  |
| (9)                             | $e^{x^2+7x-30} - 1$                           | 3.5   | 11       | <b>12</b> | 8         | 8        | 22        | 24  | 24         | 24  | 2.96      | 3.00 | 3                 |
|                                 |                                               | 3.25  | 8        | 8         | 5         | <b>6</b> | 16        | 16  | 15         | 18  | 2.81      | 3.00 |                   |
| Total (all 18 cases)            |                                               |       | 319      | 254       | 76        | 82       | 638       | 508 | 228        | 246 |           |      |                   |
| Total (excl. (1) at -0.5, -0.3) |                                               |       | 97       | 103       | 64        | 70       | 194       | 206 | 192        | 210 |           |      |                   |

extension therefore behaves as theory predicts. The cost of two Jacobian evaluations and two linear solves per iteration is justified whenever Jacobians are cheap relative to the number of iterations saved.

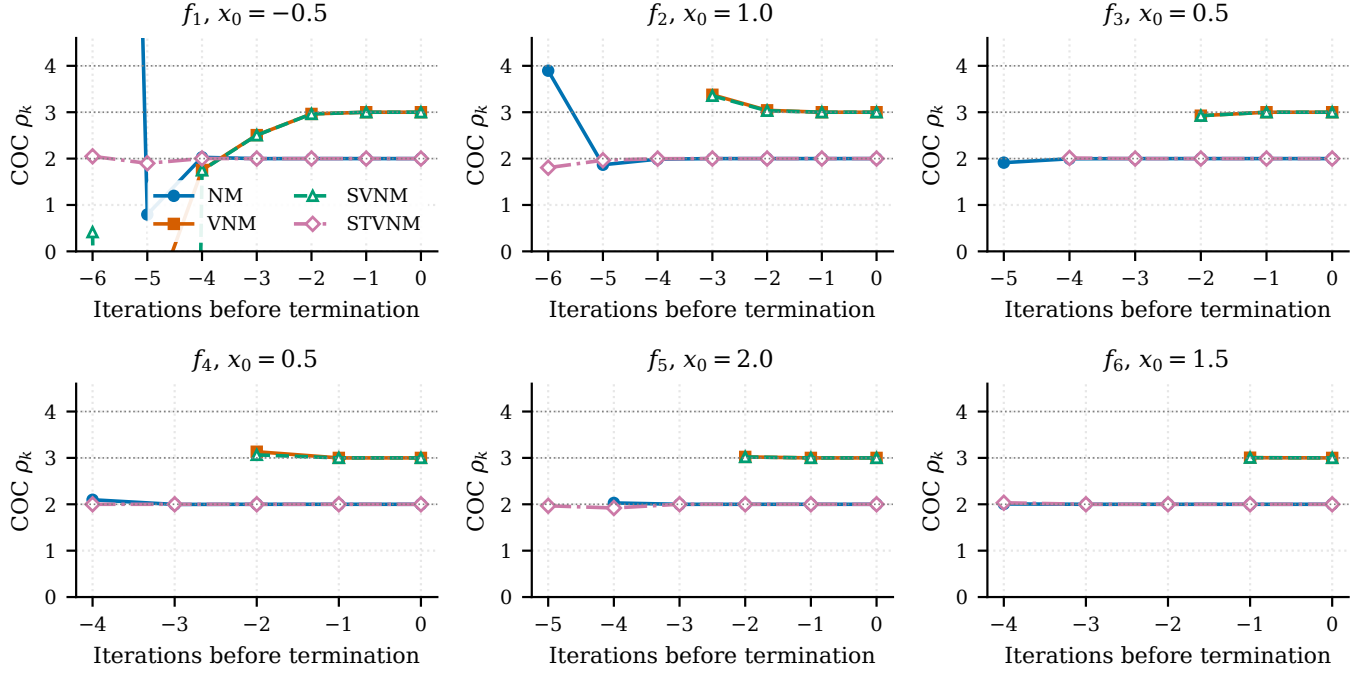
#### 4.5 Comparison with Baseline Expectations

Taking all experiments together:

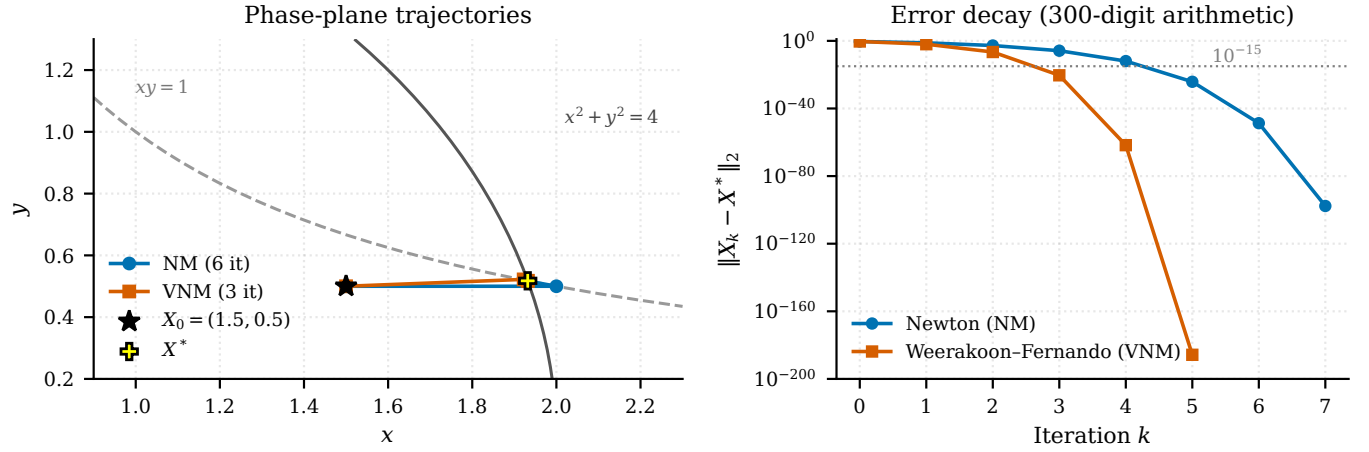
- (1) **Base paper confirmed.** VNM is third order with the constant stated by the authors, needs fewer iterations than NM

in every case, and is markedly more robust from poor starting points.

- (2) **S-VNM did not meet its design goal.** It is third order ( $E = 1.316$ ), so for these problems it is dominated by VNM ( $E = 1.442$ ).
- (3) **St-VNM did not meet its design goal.** It is second order ( $E = 1.189$ ). It remains useful only when derivatives are unavailable.



**Figure 4: Experiment C: COC per iteration in 400-digit arithmetic, aligned at the final iteration. Dotted reference lines mark orders 2, 3 and 4. VNM and S-VNM converge to 3, while NM and St-VNM converge to 2 (their markers overlap). No method approaches 4.**



**Figure 5: System (11). Left: iterates of NM and VNM from  $X_0 = (1.5, 0.5)$  in double precision (tolerance  $10^{-12}$ ). Right: error  $\|X_k - X^*\|_2$  in 300-digit arithmetic, which reveals the quadratic versus cubic decay.**

- (4) **Multivariate VNM met its design goal.** It is third order and saves 40% of Newton's iterations on the test system.

## 5 Conclusion and Future Work

*Key takeaways.* We implemented, replicated and extended the Weerakoon-Fernando variant of Newton's method. Our replication of the base paper's 18-case benchmark agrees with the published iteration counts, and our symbolic and 400-digit analyses confirm its

third-order error equation to six significant digits. The benchmarks show that VNM's practical advantage over Newton lies mainly in robustness far from the root; near the root the two methods are about equally efficient per function evaluation. A higher-order quadrature rule does not raise the order: the Simpson variant stays third order because the Newton predictor limits accuracy, and a Steffensen slope reduces the method to second order. The trapezoidal idea extends cleanly to nonlinear systems with cubic convergence.

**Table 4: Experiment C (400-digit arithmetic): asymptotic error constants  $K = |e_{n+1}|/|e_n|^p$ . The measured values agree with the theoretical predictions of Eqs. (5), (4), (7) and (9) to all six digits shown, for every entry. The COC equals 2.0000 (NM, St-VNM) or 3.0000 (VNM, S-VNM) in every case.**

|       | NM, $p=2$<br>$ C_2 $ | VNM, $p=3$<br>$ C_2^2 + \frac{1}{2}C_3 $ | S-VNM, $p=3$<br>$C_2^2$ | St-VNM, $p=2$<br>$ \frac{1}{2}f'(\alpha)C_2 $ |
|-------|----------------------|------------------------------------------|-------------------------|-----------------------------------------------|
| $f_1$ | 0.490250             | 0.270623                                 | 0.240345                | 4.04785                                       |
| $f_2$ | 0.783571             | 0.657826                                 | 0.613983                | 0.972597                                      |
| $f_3$ | 0.812547             | 0.524808                                 | 0.660233                | 0.464296                                      |
| $f_4$ | 0.220805             | 0.015214                                 | 0.048755                | 0.184771                                      |
| $f_5$ | 0.562979             | 0.361742                                 | 0.316945                | 3.14183                                       |
| $f_6$ | 0.125570             | 0.042648                                 | 0.015768                | 0.103105                                      |

We also identified a typographical error in one of the paper’s test functions.

#### Limitations.

- Only simple roots are considered; at multiple roots all methods studied degrade to linear convergence.
- NFEV treats  $f$  and  $f'$  as equally expensive, and we did not measure wall-clock time.
- The absolute stopping tolerance of  $10^{-15}$  is too strict for double precision when  $|\alpha| > 4.5$ ; a relative tolerance would be more robust.
- The benchmark is small (six scalar functions and one  $2 \times 2$  system), and we did not study basins of attraction or global convergence.
- St-VNM is not invariant to the scaling of  $f$ .
- One documentation issue remains in our code: the reference root listed for  $f_6$  in the test table (1.55714852908987) should be 1.557145598997611. This field is not used in any computation, so no result is affected.

#### Future work.

- **Genuine fourth order.** A true fourth-order upgrade requires changing the *corrector* rather than the quadrature rule. Ostrowski’s method [11] and Jarratt’s method [8] reach order 4 with three evaluations ( $E \approx 1.587$ ), which is optimal in the Kung–Traub sense [9].
- **Derivative-free third order.** Divided-difference combinations can be designed whose slope error cancels at  $O(e_n^2)$ .
- **Broader study.** Basin-of-attraction plots, modified schemes for multiple roots, and tests on large sparse systems in which Jacobian evaluations or factorisations are reused (e.g., frozen-Jacobian variants of Eq. (10)).

## Acknowledgments

We thank the course instructor of CSE 402 for guidance throughout this project. We also thank the developers of NumPy, SciPy, Matplotlib, SymPy and mpmath, on which all computations in this report rely.

## References

- [1] Richard L. Burden and J. Douglas Faires. 2011. *Numerical Analysis* (9th ed.). Brooks/Cole, Cengage Learning, Boston, MA.
- [2] Alicia Cordero and Juan R. Torregrosa. 2007. Variants of Newton’s Method Using Fifth-Order Quadrature Formulas. *Appl. Math. Comput.* 190, 1 (2007), 686–698. doi:10.1016/j.amc.2007.01.062
- [3] M. Frontini and E. Sormani. 2003. Some Variant of Newton’s Method with Third-Order Convergence. *Appl. Math. Comput.* 140, 2–3 (2003), 419–426. doi:10.1016/S0096-3003(02)00238-2
- [4] M. Frontini and E. Sormani. 2004. Third-Order Methods from Quadrature Formulae for Solving Systems of Nonlinear Equations. *Appl. Math. Comput.* 149, 3 (2004), 771–782. doi:10.1016/S0096-3003(03)00178-4
- [5] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. 2020. Array Programming with NumPy. *Nature* 585, 7825 (2020), 357–362. doi:10.1038/s41586-020-2649-2
- [6] H. H. H. Homeier. 2005. On Newton-Type Methods with Cubic Convergence. *J. Comput. Appl. Math.* 176, 2 (2005), 425–432. doi:10.1016/j.cam.2004.07.027
- [7] John D. Hunter. 2007. Matplotlib: A 2D Graphics Environment. *Computing in Science & Engineering* 9, 3 (2007), 90–95. doi:10.1109/MCSE.2007.55
- [8] P. Jarratt. 1966. Some Fourth Order Multipoint Iterative Methods for Solving Equations. *Math. Comp.* 20, 95 (1966), 434–437. doi:10.1090/S0025-5718-66-99924-8
- [9] H. T. Kung and Joseph F. Traub. 1974. Optimal Order of One-Point and Multipoint Iteration. *J. ACM* 21, 4 (1974), 643–651. doi:10.1145/321850.321860
- [10] Aaron Meurer, Christopher P. Smith, Mateusz Paprocki, et al. 2017. SymPy: Symbolic Computing in Python. *PeerJ Computer Science* 3 (2017), e103. doi:10.7717/peerj-cs.103
- [11] Alexander M. Ostrowski. 1966. *Solution of Equations and Systems of Equations* (2nd ed.). Academic Press, New York.
- [12] Ahmet Yasar Özban. 2004. Some New Variants of Newton’s Method. *Applied Mathematics Letters* 17, 6 (2004), 677–682. doi:10.1016/S0893-9659(04)90104-8
- [13] Pulseparadigm. 2026. CSE-402-Project: Implementation and Benchmarking of the Weerakoon–Fernando Variant of Newton’s Method. <https://github.com/Pulseparadigm/CSE-402-Project>. GitHub repository.
- [14] J. F. Steffensen. 1933. Remarks on Iteration. *Scandinavian Actuarial Journal* 1933, 1 (1933), 64–72. doi:10.1080/03461238.1933.10419209
- [15] The mpmath development team. 2023. mpmath: A Python Library for Arbitrary-Precision Floating-Point Arithmetic (version 1.3.0). <https://mpmath.org/>.
- [16] Joseph F. Traub. 1964. *Iterative Methods for the Solution of Equations*. Prentice-Hall, Englewood Cliffs, NJ.
- [17] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. 2020. SciPy 1.0: Fundamental Algorithms for Scientific Computing in Python. *Nature Methods* 17 (2020), 261–272. doi:10.1038/s41592-019-0686-2
- [18] Sunethra Weerakoon and T. G. I. Fernando. 2000. A Variant of Newton’s Method with Accelerated Third-Order Convergence. *Applied Mathematics Letters* 13, 8 (2000), 87–93. doi:10.1016/S0893-9659(00)00100-2

## A Reproducibility

The folder code/ accompanying this report contains:

- `weerakoon_fernando.py`: the project module (all five methods and Experiment A); run it with `python weerakoon_fernando.py`.
- `replicate_table1.py`: Experiment B; run it with `python replicate_table1.py` or.
- `verify_orders.py`: Experiment C (400-digit COC and error constants).
- `make_figures.py`: regenerates every figure in `figures/`.

Requirements: Python  $\geq 3.8$ , NumPy, SciPy, Matplotlib and mpmath (and SymPy for the symbolic series). The full source history is available in the project repository [13].